Computer Networks- Socket Programming: Multi-Client Online Quiz System with Real-Time Ranking

Team 2
Krishna Tejas Voruganti - PES1UG24AM436
Sanjit C - PES1UG24AM448
Haneesh Elamanti - PES1UG24AM471

Project Details:
Time-bound question handling
Real-time leaderboard updates
Synchronization of quiz rounds
Latency and fairness evaluation
The project uses TCP sockets directly and SSL/TLS-based secure communication for all control and data exchanges
The application supports multiple concurrent clients
Communication must occufs over the network using sockets.

Client code:

```
import socket
import threading
import json
import time
import sys

SERVER_HOST = "127.0.0.1"
SERVER_PORT = 5000


def send_json(sock, obj):
    data = json.dumps(obj).encode("utf-8") + b"\n"
    sock.sendall(data)


def receiver_loop(sock):
    buf = b""
    while True:
        try:
```

```
            data = sock.recv(4096)
            if not data:
                print("[INFO] Disconnected from server.")
                break
            buf += data
            while b"\n" in buf:
                line, buf = buf.split(b"\n", 1)
                line = line.strip()
                if not line:
                    continue
                try:
                    msg = json.loads(line.decode("utf-8"))
                    handle_server_msg(sock, msg)
                except json.JSONDecodeError:
                    continue
        except OSError:
            break


def handle_server_msg(sock, msg):
    t = msg.get("type")
    if t == "info":
        print(f"[INFO] {msg.get('msg')}")
    elif t == "question":
        qid = msg["qid"]
        print("\n" + "=" * 40)
        print(f"Question {qid} (time limit: {msg['time_limit']}s):")
        print(msg["text"])
        for opt in msg["options"]:
            print(opt)
        print("Enter your choice (A/B/C/D) and press Enter:")

        # Take input in the main thread for simplicity.
        # Server enforces time limit; late answers are discarded.
        def get_answer():
            choice = sys.stdin.readline().strip().upper()
            if choice not in {"A", "B", "C", "D"}:
                print("[WARN] Invalid choice, ignoring.")
                return
            send_json(
                sock,
                {
                    "type": "answer",
```

```
                    "qid": qid,
                    "choice": choice,
                    "client_send_ts": time.time(),
                },
            )

        threading.Thread(target=get_answer, daemon=True).start()

    elif t == "leaderboard":
        print("\n--- Leaderboard after question", msg["qid"], "---")
        for entry in msg["entries"]:
            print(f"{entry['name']}: {entry['score']}")
    elif t == "quiz_end":
        print("\n=== FINAL STANDINGS ===")
        for entry in msg["entries"]:
            print(f"{entry['name']}: {entry['score']}")
        print("Quiz finished. Press Ctrl+C to exit.")
    else:
        print("[DEBUG] Unknown message from server:", msg)


def main():
    name = input("Enter your nickname: ").strip() or "Player"

    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((SERVER_HOST, SERVER_PORT))
    print(f"[INFO] Connected to {SERVER_HOST}:{SERVER_PORT}")

    # Send join message
    send_json(sock, {"type": "join", "name": name})

    threading.Thread(target=receiver_loop, args=(sock,),
daemon=True).start()

    # Keep main thread alive
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\n[INFO] Exiting...")
    finally:
        sock.close()
```

```
if __name__ == "__main__":
    main()
```

Server Code:

```
import socket
import threading
import json
import time
import queue

HOST = "0.0.0.0"
PORT = 5000

# Simple quiz database
QUIZ_QUESTIONS = [
    {
        "qid": 1,
        "text": "What is the capital of France?",
        "options": ["A) Berlin", "B) Madrid", "C) Paris", "D) Rome"],
        "correct": "C",
        "time_limit": 10.0,  # seconds
    },
    {
        "qid": 2,
        "text": "2 + 2 * 3 = ?",
        "options": ["A) 8", "B) 10", "C) 12", "D) 6"],
        "correct": "A",
        "time_limit": 8.0,
    },
    {
        "qid": 3,
        "text": "TCP stands for?",
        "options": [
            "A) Transmission Control Protocol",
            "B) Transport Core Protocol",
            "C) Transfer Channel Protocol",
            "D) Transmission Channel Path",
        ],
        "correct": "A",
        "time_limit": 8.0,
    },
```

```
]

# Global state (protected by locks where needed)
clients_lock = threading.Lock()
clients = {}  # conn -> {"name": str, "score": int, "latencies":
[float]}

message_queue = queue.Queue()

# Per-question bookkeeping
question_send_times = {}  # (conn, qid) -> send_ts


def send_json(conn, obj):
    try:
        data = json.dumps(obj).encode("utf-8") + b"\n"
        conn.sendall(data)
    except OSError:
        # Socket likely closed
        pass


def broadcast(obj):
    with clients_lock:
        for conn in list(clients.keys()):
            send_json(conn, obj)


def handle_client(conn, addr):
    buf = b""
    try:
        while True:
            data = conn.recv(4096)
            if not data:
                break
            buf += data
            while b"\n" in buf:
                line, buf = buf.split(b"\n", 1)
                line = line.strip()
                if not line:
                    continue
                try:
                    msg = json.loads(line.decode("utf-8"))
```

```
                    message_queue.put((conn, msg, time.time()))
                except json.JSONDecodeError:
                    # Ignore malformed
                    continue
    finally:
        # Remove client on disconnect
        with clients_lock:
            info = clients.pop(conn, None)
        if info is not None:
            print(f"[INFO] {info['name']} disconnected from {addr}")
        conn.close()


def wait_for_players(min_players=1, wait_time=10.0):
    start = time.time()
    while time.time() - start < wait_time:
        with clients_lock:
            if len(clients) >= min_players:
                return True
        time.sleep(0.5)
    return False


def run_quiz():
    print("[INFO] Waiting for players...")
    ready = wait_for_players(min_players=1, wait_time=30.0)
    if not ready:
        print("[INFO] Not enough players, aborting quiz.")
        return

    broadcast({"type": "info", "msg": "Quiz is starting!"})

    # Process questions sequentially
    for q in QUIZ_QUESTIONS:
        qid = q["qid"]
        correct = q["correct"]
        time_limit = q["time_limit"]

        # Broadcast question
        with clients_lock:
            now = time.time()
            for conn in clients.keys():
                question_send_times[(conn, qid)] = now
```

```
                send_json(
                    conn,
                    {
                        "type": "question",
                        "qid": qid,
                        "text": q["text"],
                        "options": q["options"],
                        "time_limit": time_limit,
                    },
                )

        print(f"[INFO] Sent question {qid}")
        deadline = now + time_limit

        # Collect answers until deadline
        answers = {}  # conn -> (choice, recv_ts, client_send_ts)

        while time.time() < deadline:
            remaining = deadline - time.time()
            if remaining <= 0:
                break
            try:
                conn, msg, recv_ts = message_queue.get(timeout=min(0.5,
remaining))
            except queue.Empty:
                continue

            if msg.get("type") == "answer" and msg.get("qid") == qid:
                choice = msg.get("choice")
                client_send_ts = msg.get("client_send_ts", recv_ts)
                if conn not in answers:
                    answers[conn] = (choice, recv_ts, client_send_ts)

        # Score answers and update latency stats
        with clients_lock:
            for conn, (choice, recv_ts, client_send_ts) in
answers.items():
                info = clients.get(conn)
                if info is None:
                    continue
                q_send_ts = question_send_times.get((conn, qid))
                if q_send_ts is None:
                    continue
```

```
                response_time = recv_ts - q_send_ts
                approx_oneway = recv_ts - client_send_ts

                info.setdefault("latencies", []).append(response_time)

                # Only accept if within time limit (server clock)
                if recv_ts <= deadline and choice == correct:
                    info["score"] += 10

                print(
                    f"[DEBUG] {info['name']} q{qid} ans={choice} "
                    f"resp_time={response_time:.3f}s,
approx_oneway={approx_oneway:.3f}s"
                )

        # Broadcast leaderboard
        with clients_lock:
            leaderboard = sorted(
                [
                    {"name": v["name"], "score": v["score"]}
                    for v in clients.values()
                ],
                key=lambda x: (-x["score"], x["name"]),
            )
        broadcast({"type": "leaderboard", "qid": qid, "entries":
leaderboard})

    # End quiz and show final standings
    with clients_lock:
        final_board = sorted(
            [{"name": v["name"], "score": v["score"]} for v in
clients.values()],
            key=lambda x: (-x["score"], x["name"]),
        )
    broadcast({"type": "quiz_end", "entries": final_board})

    print("[INFO] Quiz ended. Final standings:")
    for entry in final_board:
        print(f"  {entry['name']}: {entry['score']}")

    # Optional: print latency stats
    with clients_lock:
```

```
        for conn, info in clients.items():
            lats = info.get("latencies", [])
            if lats:
                avg_lat = sum(lats) / len(lats)
                print(
                    f"  {info['name']} avg response time:
{avg_lat:.3f}s "
                    f"(n={len(lats)})"
                )


def accept_loop(server_sock):
    while True:
        conn, addr = server_sock.accept()
        print(f"[INFO] Connection from {addr}")
        # Temporary name until JOIN received
        with clients_lock:
            clients[conn] = {"name": f"{addr}", "score": 0,
"latencies": []}
        threading.Thread(target=handle_client, args=(conn, addr),
daemon=True).start()


def main():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((HOST, PORT))
        s.listen()
        print(f"[INFO] Server listening on {HOST}:{PORT}")

        threading.Thread(target=accept_loop, args=(s,),
daemon=True).start()

        # Wait for join messages and set nicknames
        def join_handler():
            while True:
                conn, msg, recv_ts = message_queue.get()
                if msg.get("type") == "join":
                    name = msg.get("name", "Player")
                    with clients_lock:
                        if conn in clients:
                            clients[conn]["name"] = name
```

```
                    send_json(conn, {"type": "info", "msg": f"Welcome,
{name}!"})
                else:
                    # Non-join messages go back into the queue for quiz
loop to process
                    message_queue.put((conn, msg, recv_ts))
                    time.sleep(0.01)  # avoid busy looping

        threading.Thread(target=join_handler, daemon=True).start()

        # Run one quiz; you can loop this or restart as needed
        run_quiz()

        # Keep server alive for inspection; in real code, you may exit
or restart quiz
        while True:
            time.sleep(1)


if __name__ == "__main__":
    main()
```

Output:

Server code output

```
C:\Users\Krish\Desktop\Krishna\Study\Engineering\Sem 4\CN\Lab\Jackfruit>python server.py
[INFO] Server listening on 0.0.0.0:5000
[INFO] Waiting for players...
[INFO] Connection from ('127.0.0.1', 51876)
[INFO] Sent question 1
[INFO] Connection from ('127.0.0.1', 51878)
[INFO] Sent question 2
[DEBUG] Sanjit q2 ans=A resp_time=5.972s, approx_oneway=0.000s
[INFO] Sent question 3
[DEBUG] Sanjit q3 ans=A resp_time=4.125s, approx_oneway=0.000s
[INFO] Quiz ended. Final standings:
  Sanjit: 20
  Krishna: 0
  Sanjit avg response time: 5.048s (n=2)
```

Client code: output:

```
C:\Users\Krish\Desktop\Krishna\Study\Engineering\Sem 4\CN\Lab\Jackfruit>python client.py
Enter your nickname: Krishna
[INFO] Connected to 127.0.0.1:5000
[INFO] Welcome, Krishna!
[INFO] Quiz is starting!

========================================
Question 1 (time limit: 10.0s):
What is the capital of France?
A) Berlin
B) Madrid
C) Paris
D) Rome
Enter your choice (A/B/C/D) and press Enter:

--- Leaderboard after question 1 ---
Krishna: 0
Sanjit: 0

========================================
Question 2 (time limit: 8.0s):
2 + 2 * 3 = ?
A) 8
B) 10
C) 12
D) 6
Enter your choice (A/B/C/D) and press Enter:
a

--- Leaderboard after question 2 ---
Sanjit: 10
Krishna: 0

========================================
Question 3 (time limit: 8.0s):
TCP stands for?
A) Transmission Control Protocol
B) Transport Core Protocol
C) Transfer Channel Protocol
D) Transmission Channel Path
Enter your choice (A/B/C/D) and press Enter:
b

--- Leaderboard after question 3 ---
Sanjit: 20
Krishna: 0

=== FINAL STANDINGS ===
Sanjit: 20
Krishna: 0
Quiz finished. Press Ctrl+C to exit.
```